02244 Logic for Security
Information Flow
Week 11: Information Leakage

Sebastian Mödersheim

April 27, 2026

About the Assignment 2

Many people have asked:

Do we have to implement the server?

Answer: NO! You should give a concept/design for a server:

- What security labels does it use (maybe user-definend)
- What API of commands does the server offer to the user
- What happens when these commands are executed
  - ★ This can be a short piece of (pseudo-) code or even plain English/Danish trying to be precise about what happens.
  - ★ E.g. for the uploaded code we perform the following information flow analysis (Approach, security lattice,...)
- Thus: how the server orchestrates the activities/security labels
- A security argument:
  - ★ what exactly are the security policies given by the labels/users?
  - ★ why can the server be sure that no security policies are ever violated? (e.g., use non-interference arguments)

Challenge

```
testrecord  = record [ subject : cpr  {shs:shs},
                       positive: bool {shs:shs} ]
publish_stat(db:array[testrecord:{⊥}]{⊥})
returns infections:int{⊥}

lookup_result(db:array[testrecord:{⊥}]{⊥},
              client:cpr{⊥})
returns result:bool{client:client}
```

`shs` is for sundhedsstyrelsen
Implement the functions `publish_stat` (giving total number of infections, i.e., positive testrecords) and `lookup_result` (giving the result of the specified client):

- What authority/declassifications do the functions need?
- Prove that this is indeed safe.

Challenge

```
testrecord  = record[subject:cpr{shs:shs},positive:bool{shs:shs}]
publish_stat(db:array[testrecord:{⊥}]{⊥})
returns infections:int{⊥}{
  count:int {shs:shs} = 0;
  i:int{⊥} = 0;
  while (i<db.length){
    if (db[i].positive)
      count=count+1;

    i=i+1;

  }
  if_acts_for(publish_stat,shs){
    infections = declassify(count,{⊥});




  }
}
```

Challenge

```
testrecord  = record[subject:cpr{shs:shs},positive:bool{shs:shs}]
publish_stat(db:array[testrecord:{⊥}]{⊥})
returns infections:int{⊥}{
  count:int {shs:shs} = 0;
  i:int{⊥} = 0;
  while (i<db.length){
    if (db[i].positive)
      count=count+1; // Infoflow: db[i].positive,i,db.length --> count
                     // OK: {⊥},{shs:shs} <= {shs:shs}
    i=i+1; // Infoflow: i,db.length --> i
           // OK: {⊥} <= {⊥}
  }
  if_acts_for(publish_stat,shs){
    infections = declassify(count,{⊥});
    // Infoflow: count-->infections not allowed without declassification
    // declassif. count:{shs:shs} to {⊥} is allowed when acting as shs
    // Infoflow: declassify(count,{⊥}) -> infections
    // OK: {⊥} <= {⊥}
  }
}
```

Challenge

```
testrecord  = record[subject:cpr{shs:shs},positive:bool{shs:shs}]
lookup_result(db:array[testrecord:{⊥}]{⊥},client:cpr{⊥})
returns result:bool{client:client}{
  int:i{⊥} = 0;
  tmp_result:bool{shs:shs} = false;
  while (i<db.length){
    if (db[i].subject=client){
      tmp_result=db[i].positive;




    }
    i = i+1; // similar as in publish_stat
  }
  if_acts_for(lookup_result,shs){
    result := declassify(tmp_result,{⊥});




  }
}
```

Challenge

```
testrecord  = record[subject:cpr{shs:shs},positive:bool{shs:shs}]
lookup_result(db:array[testrecord:{⊥}]{⊥},client:cpr{⊥})
returns result:bool{client:client}{
  int:i{⊥} = 0;
  tmp_result:bool{shs:shs} = false;
  while (i<db.length){
    if (db[i].subject=client){
      tmp_result=db[i].positive;
      // Infoflow: i,db.length,db[i].subject,client,db[i].positive
      //           --> tmp_result
      // OK: {⊥},{shs:shs} <= {shs:shs}
    }
    i = i+1; // similar as in publish_stat
  }
  if_acts_for(lookup_result,shs){
    result := declassify(tmp_result,{⊥});
    // IF: tmp_result-->result not allowed without declassification
    // decl. tmp_result:{shs:shs} to {⊥} is allowed when acting as shs
    // Infoflow: declassify(tmp_result,{⊥}) -> result
    // OK: {⊥} <= {client:client}
  }
}
```

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$
  - ★ Is this sufficient?

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$
  - ★ Is this sufficient?
- No: Implicit flows! Example:
  `if y>0 then x := t` with $\underline{x} = L, \underline{y} = H$

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$
  - ★ Is this sufficient?
- No: Implicit flows! Example:
  `if y>0 then x := t` with $\underline{x} = L, \underline{y} = H$
  - ★ Suppose the interpreter remembers the context: all security labels of conditions under which the current command is (here $\underline{y} = H$)
  - ★ If writing into a lower variable (here $\underline{x} = L$), we stop execution.

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$
  - ★ Is this sufficient?
- No: Implicit flows! Example:
  `if y>0 then x := t` with $\underline{x} = L, \underline{y} = H$
  - ★ Suppose the interpreter remembers the context: all security labels of conditions under which the current command is (here $\underline{y} = H$)
  - ★ If writing into a lower variable (here $\underline{x} = L$), we stop execution.
  - ★ That stopping is observable and thus leaks $y > 0$!

Monitoring Information Flow

So far, we have used information flow as a static analysis

- A compiler checks the information flow at compile time

What about an interpreter that checks it at run time?

- At any step monitoring that no illegal information is flowing, otherwise stopping execution.
- Consider a step $x := t$,
  - ★ while computing t, compute the security label $\underline{t}$.
  - ★ before writing the result into x, check $\underline{x} \sqsubseteq \underline{t}$
  - ★ Is this sufficient?
- No: Implicit flows! Example:
  `if y>0 then x := t` with $\underline{x} = L, \underline{y} = H$
  - ★ Suppose the interpreter remembers the context: all security labels of conditions under which the current command is (here $\underline{y} = H$)
  - ★ If writing into a lower variable (here $\underline{x} = L$), we stop execution.
  - ★ That stopping is observable and thus leaks $y > 0$!
- So: NO, you need to make a bit of static analysis to identify implicit flows before they happen.

Timing Leaks

Consider the following naïve algorithm for $b^x \mod N$:

```
finished=false;
result=1;
while(x>0){
  result=result*b mod N;
  x=x-1;
}
finished=true
```

- Assume `finished` is level Low, while all other variables are High.
- This satisfies classical information flow.

Timing Leaks

Consider the following naïve algorithm for $b^x \mod N$:

```
finished=false;
result=1;
while(x>0){
  result=result*b mod N;
  x=x-1;
}
finished=true
```

- Assume `finished` is level Low, while all other variables are High.
- This satisfies classical information flow.
- However, an intruder who can observe `finished` can estimate x — if they have a clock.
- This can be a problem in cryptographic algorithms, although the above example would never be used in crypto. (Why?)

Timing Leaks

Consider the following less naïve algorithm for $b^x \mod N$:

```
result=1;
while(x>1){
  if (x%2==0){x=x/2;}
  else{
    x=(x-1)/2;
    result=result*b % N;
  }
  b=b*b % N;
}
result=result*b % N;
```

Timing Leaks

Consider the following less naïve algorithm for $b^x \mod N$:

```
result=1;
while(x>1){
  if (x%2==0){x=x/2;}
  else{
    x=(x-1)/2;
    result=result*b % N;
  }
  b=b*b % N;
}
result=result*b % N;
```

- The runtime still depends on `x`
  - ★ position of the most significant set bit in `x`
  - ★ number of bits set in `x`

Timing Leaks

Consider the following less naïve algorithm for $b^x \mod N$:

```
result=1;
while(x>1){
  if (x%2==0){x=x/2;}
  else{
    x=(x-1)/2;
    result=result*b % N;
  }
  b=b*b % N;
}
result=result*b % N;
```

- The runtime still depends on `x`
  - ★ position of the most significant set bit in `x`
  - ★ number of bits set in `x`
- It is not difficult to make this algorithm `constant time`:
  - ★ Ensure that also the `then` case performs a modular multiplication.
  - ★ Ensure that there are exactly η rounds of the `while` loop

    where η is the number of bits of `x`.

A Remote Timing Attack on RSA

As a more realistic example for timing leaks in cryptography, we consider now the paper

[1] D. Brumley and D. Boneh: *Remote timing attacks are practical*, Comput. Networks, 48(5), pp. 701–716, 2005.

- We skip some parts
- We make some simplifications
- You do not need to make any such analysis in your report!

Recall: RSA

Keygeneration:

- Choose bit-size η (recommended: $\eta \geq 2048$)
- Choose random `primes` p, q of size $\frac{\eta}{2}$.
- Let $N := p \cdot q$
- Choose random e of size η
- Compute $d := e^{-1} \mod (p-1)(q-1)$
- The resulting public key is (N, e)
  - ★ in OFMC for instance $\mathrm{pk}(B)$
- The resulting private key is (p, q, d)
  - ★ in OFMC for instance: $\mathrm{inv}(\mathrm{pk}(B))$

Encryption:

- $c := m^e \mod N$
  - ★ in OFMC for instance: $\{m\}_{\mathrm{pk}(B)}$

Decryption:

- $m := c^d \mod N$
  - ★ in OFMC: decryption handled as part of the Dolev-Yao rules

Recall: RSA

Keygeneration:

- Choose bit-size η (recommended: $\eta \geq 2048$)
- Choose random `primes` p, q of size $\frac{\eta}{2}$.
- Let $N := p \cdot q$
- Choose random e of size η
- Compute $d := e^{-1} \mod (p-1)(q-1)$
- The resulting public key is (N, e)
  - ★ in OFMC for instance pk(B)
- The resulting private key is (p, q, d)
  - ★ in OFMC for instance: inv(pk(B))

Encryption: **to prevent guessing and multiplication attacks**

- $c := \boxed{\text{pad,m,rand}}^e \mod N$
  - ★ where *pad* is a fixed padding string, *rand* is a fresh random string

Decryption:

- $\boxed{\text{pad,m,rand}} := c^d \mod N$
  - ★ if pad is the fixed padding string, return m, otherwise error.

Montgomery Representation

- Computing the $\cdot \mod N$ operation is very expensive, but is needed up to 2η times during an exponentiation.
- Idea: with a different representation of numbers, we can reduce it to one single $\cdot \mod N$ operation!

Montgomery Representation (MR)

- ★ Let $R = 2^\eta$
- ★ Represent a number a as $\boxed{a \cdot R \mod N}$.
- ★ Note: while $a \cdot R$ has 2η bits, modulo N we have again η bits.
- ★ We just write $\boxed{a \cdot R}$ in the following for short.

Multiplication modulo N in MR

Multiplying two numbers a and b in MR:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

Multiplication modulo N in MR

Multiplying two numbers a and b in MR:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Now the result must be divided by R to obtain $\boxed{a \cdot b \cdot R}$, i.e., the MR of $a \cdot b$.

Multiplication modulo N in MR

Multiplying two numbers a and b in MR:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Now the result must be divided by R to obtain $\boxed{a \cdot b \cdot R}$, i.e., the MR of $a \cdot b$.
- Suppose:

$$\underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}} = \underbrace{\boxed{\text{XY}}}_{\eta \text{ bits}} \underbrace{\boxed{0\ldots0}}_{\eta \text{ bits}} \tag{1}$$

  ★ then the division by R would just be $\underbrace{\boxed{\text{XY}}}_{\eta \text{ bits}}$.

Multiplication modulo N in MR

Multiplying two numbers a and b in MR:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Now the result must be divided by R to obtain $\boxed{a \cdot b \cdot R}$, i.e., the MR of $a \cdot b$.
- Suppose:

$$\underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}} = \underbrace{\boxed{\text{XY}}}_{\eta \text{ bits}} \underbrace{\boxed{0\ldots0}}_{\eta \text{ bits}} \qquad (1)$$

  - ★ then the division by R would just be $\underbrace{\boxed{\text{XY}}}_{\eta \text{ bits}}$.

- But how to achieve (1)?

Multiplication modulo N in MR

- It is nearly certain that the lower η bits of $\boxed{a \cdot b \cdot R \cdot R}$ are not all zero.
- But note that $a \equiv_N a + N$, i.e. adding the modulus any number of times does not change the result modulo N.

Multiplication modulo N in MR

- It is nearly certain that the lower η bits of $\boxed{a \cdot b \cdot R \cdot R}$ are not all zero.
- But note that $a \equiv_N a + N$, i.e. adding the modulus any number of times does not change the result modulo N.
- For instance say $N = \langle 101 \rangle_2$

a	1	0	1	1	0	1	0	0
$+4N$	0	0	0	1	0	1	0	0
$\equiv_N a$	1	1	0	0	1	0	0	0

Multiplication modulo N in MR

- It is nearly certain that the lower η bits of $\boxed{a \cdot b \cdot R \cdot R}$ are not all zero.
- But note that $a \equiv_N a + N$, i.e. adding the modulus any number of times does not change the result modulo N.
- For instance say $N = \langle 101 \rangle_2$

a	1	0	1	1	0	1	0	0
$+4N$	0	0	0	1	0	1	0	0
$\equiv_N a$	1	1	0	0	1	0	0	0

- In this way we can achieve the lower half to be zero without changing the result modulo N.

Multiplication modulo N in MR

So we now have:

- Step 1:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Step 2:

$$\underbrace{\boxed{a \cdot b \cdot R \cdot R + k \cdot N}}_{2\eta \text{ bits}} = \underbrace{\boxed{XY}}_{\eta \text{ bits}} \underbrace{\boxed{0...0}}_{\eta \text{ bits}}$$

- Step 3: $\boxed{a \cdot b \cdot R} \equiv_N \underbrace{\boxed{XY}}_{\eta \text{ bits}}$

Multiplication modulo N in MR

So we now have:

- Step 1:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Step 2:

$$\underbrace{\boxed{a \cdot b \cdot R \cdot R + k \cdot N}}_{2\eta \text{ bits}} = \underbrace{\boxed{\mathrm{XY}}}_{\eta \text{ bits}} \underbrace{\boxed{0 \ldots 0}}_{\eta \text{ bits}}$$

- Step 3: $\boxed{a \cdot b \cdot R} \equiv_N \underbrace{\boxed{\mathrm{XY}}}_{\eta \text{ bits}}$
- But there is one small problem: XY may be larger than N (but not larger than $2N$)
  - ★ So Step 4: if $\mathrm{XY} > N$ subtract N from it!

Multiplication modulo N in MR

So we now have:

- Step 1:

$$\underbrace{\boxed{a \cdot R}}_{\eta \text{ bits}} \cdot \underbrace{\boxed{b \cdot R}}_{\eta \text{ bits}} \equiv_N \underbrace{\boxed{a \cdot b \cdot R \cdot R}}_{2\eta \text{ bits}}$$

- Step 2:

$$\underbrace{\boxed{a \cdot b \cdot R \cdot R + k \cdot N}}_{2\eta \text{ bits}} = \underbrace{\boxed{\mathrm{XY}}}_{\eta \text{ bits}} \underbrace{\boxed{0 \ldots 0}}_{\eta \text{ bits}}$$

- Step 3: $\boxed{a \cdot b \cdot R} \equiv_N \underbrace{\boxed{\mathrm{XY}}}_{\eta \text{ bits}}$
- But there is one small problem: XY may be larger than N (but not larger than $2N$)
  - ★ So Step 4: if $\mathrm{XY} > N$ subtract N from it!
- This last step is a timing leak!
  - ★ In the exponentation $c^d \mod N$, this is happening more often if c is close to N, and that can give a measurable difference!

Chinese Remainder Theorem

- So the exponentiation with MR can leak the modulus N.
- N in RSA is public, however, upon decryption, the factors $p \cdot q = N$ are known.
- Implementations usually exploit this using the Chinese Remainder Theorem:
  - ★ One can compute $c^d \mod N$ from the smaller problems
    - $c^{d \mod p} \mod p$
    - $c^{d \mod q} \mod q$
  - ★ This essentially makes it possible to obtain q by a timing attack, and thus the rest of the private key.
    - The paper uses also another timing leak in the multiplication.

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

- Suppose the intruder already knows the a few of the most significant bits of q. Then the next bit can be obtained like this:

$$\begin{array}{lcccccccc} g & = & b_1 & b_2 & b_3 & 0 & 0 & 0 & 0 & 0 \\ g_{hi} & = & b_1 & b_2 & b_3 & 1 & 0 & 0 & 0 & 0 \end{array}$$

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

- Suppose the intruder already knows the a few of the most significant bits of q. Then the next bit can be obtained like this:

$$\begin{array}{lcccccccc} g & = & b_1 & b_2 & b_3 & 0 & 0 & 0 & 0 & 0 \\ g_{hi} & = & b_1 & b_2 & b_3 & 1 & 0 & 0 & 0 & 0 \end{array}$$

- Then q is either (a) between g and g_{hi} or (b) above g_{hi}.

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

- Suppose the intruder already knows the a few of the most significant bits of q. Then the next bit can be obtained like this:

$$\begin{array}{lcccccccc} g & = & b_1 & b_2 & b_3 & 0 & 0 & 0 & 0 & 0 \\ g_{hi} & = & b_1 & b_2 & b_3 & 1 & 0 & 0 & 0 & 0 \end{array}$$

- Then q is either (a) between g and g_{hi} or (b) above g_{hi}.
- Measure the time for $c = g$ and $c = g_{hi}$ (under some transf.)

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

- Suppose the intruder already knows the a few of the most significant bits of q. Then the next bit can be obtained like this:

$$\begin{array}{lcccccccc} g & = & b_1 & b_2 & b_3 & 0 & 0 & 0 & 0 & 0 \\ g_{hi} & = & b_1 & b_2 & b_3 & 1 & 0 & 0 & 0 & 0 \end{array}$$

- Then q is either (a) between g and g_{hi} or (b) above g_{hi}.
- Measure the time for $c = g$ and $c = g_{hi}$ (under some transf.)
- Timing reliably tells whether (a) or (b) is the case.
  - ★ This is a bit tricky, see original paper...

The Timing Attack

Scenario: server running OpenSSL:

$$A \to B : \{PMS\}_{\mathsf{pk}(B)}, \ldots \quad (PMS \text{ is a fresh key})$$

The intruder can send an arbitrary value c and B will try to decrypt it. What to send as c?

- Suppose the intruder already knows the a few of the most significant bits of q. Then the next bit can be obtained like this:

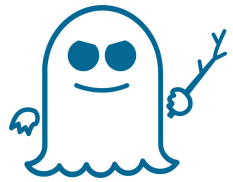
$$\begin{array}{lcccccccc} g & = & b_1 & b_2 & b_3 & 0 & 0 & 0 & 0 & 0 \\ g_{hi} & = & b_1 & b_2 & b_3 & 1 & 0 & 0 & 0 & 0 \end{array}$$

- Then q is either (a) between g and g_{hi} or (b) above g_{hi}.
- Measure the time for $c = g$ and $c = g_{hi}$ (under some transf.)
- Timing reliably tells whether (a) or (b) is the case.
  - ★ This is a bit tricky, see original paper...
- Compute the next bit...

Counter-Measures

- Recommendations [1]:
  - ★ blinding: multiply c with random r before decryption (and divide result by r afterwards)
  - ★ constant time: ensure runtime does not depend on the data.
- Compiler Design with special forms of information flow can help here:
  - ★ Generate Code that does not have timing leaks.
  - ★ Compiler ensures that optimizations are not introducing time leaks.
  - ★ Target all popular processors.
  - ★ Verified implementation (no gap to mathematical model)
- Project Everest: https://project-everest.github.io
- **There is (almost) no excuse for using unverified crypto implementations!**

Spectre



Another spooky timing/side channel attack:

[3] Paul Kocher et al.: *Spectre attacks: exploiting speculative execution*, Commun. ACM, 63(7), pp. 93–101, 2020.

- Exploit combines two common performance optimizations of modern processors.
  - ★ Caching
  - ★ Speculative execution.
- Essentially: using speculative execution to leave "traces" in the cache and obtain these traces.
- Works on a wide variety of processors.

Speculative Execution

```
a=3;
b=Array[6];
c=f(a);
```

- Accessing `Array[6]` may be slow if not in cache.
- Instead of waiting, we could start computing `f(a)`.

Speculative Execution

```
a=3;
if (Array[6]>3)
  c=f(a);
else
  c=g(a);
```

- If `Array[6]` takes time, we do not know for some time what is the next command.

Speculative Execution

```
a=3;
if (Array[6]>3)
  c=f(a);
else
  c=g(a);
```

- If `Array[6]` takes time, we do not know for some time what is the next command.
- Branch prediction: if we know `Array[6]>3` has been often true in the past, we could speculate it is going to be true again.

Speculative Execution

```
a=3;
if (Array[6]>3)
  c=f(a);
else
  c=g(a);
```

- If `Array[6]` takes time, we do not know for some time what is the next command.
- Branch prediction: if we know `Array[6]>3` has been often true in the past, we could speculate it is going to be true again.
  - ★ Go ahead computing `f(a)`
  - ★ If `Array[6]>3` turns out to be true, we win.
  - ★ Otherwise, we have to revert the processor, but we did not loose anything.

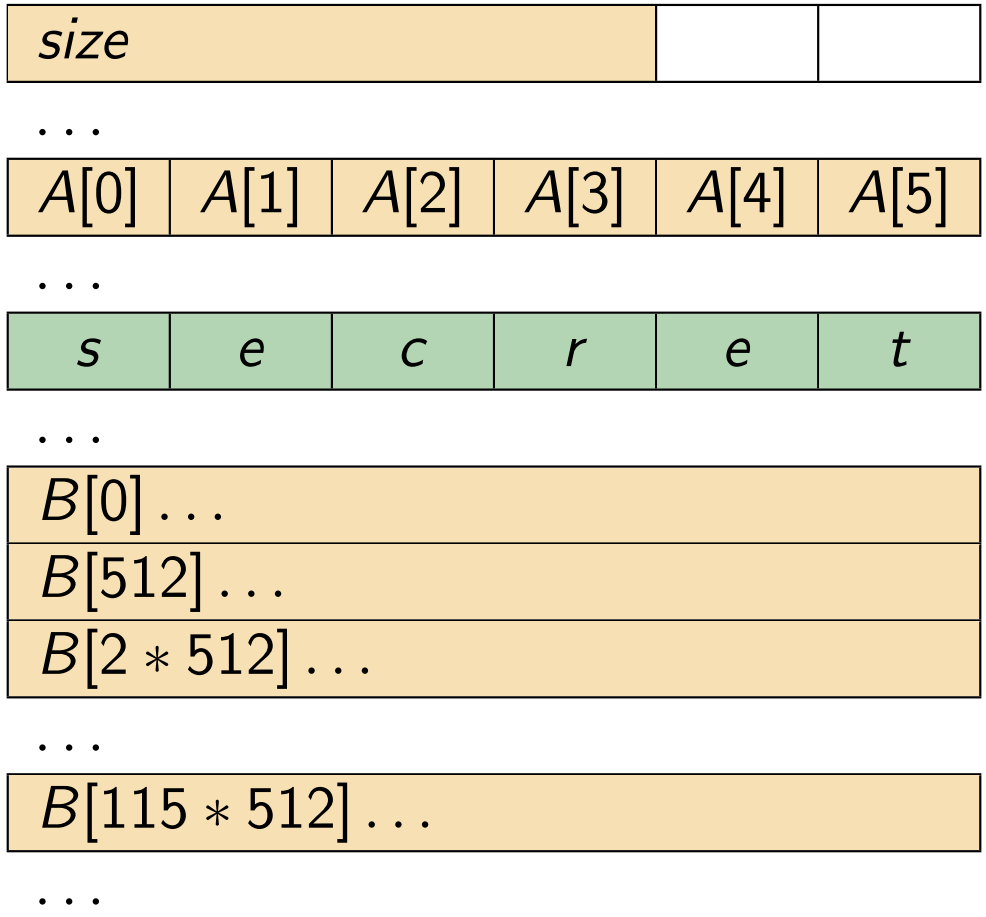
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Dramatis personae:

- `y`: chosen by the intruder.
- `A`: array of size `size`.
- The intruder is allowed to know arrays `A` and `B`.
- There is a *secret* in the application memory somewhere after `A`.



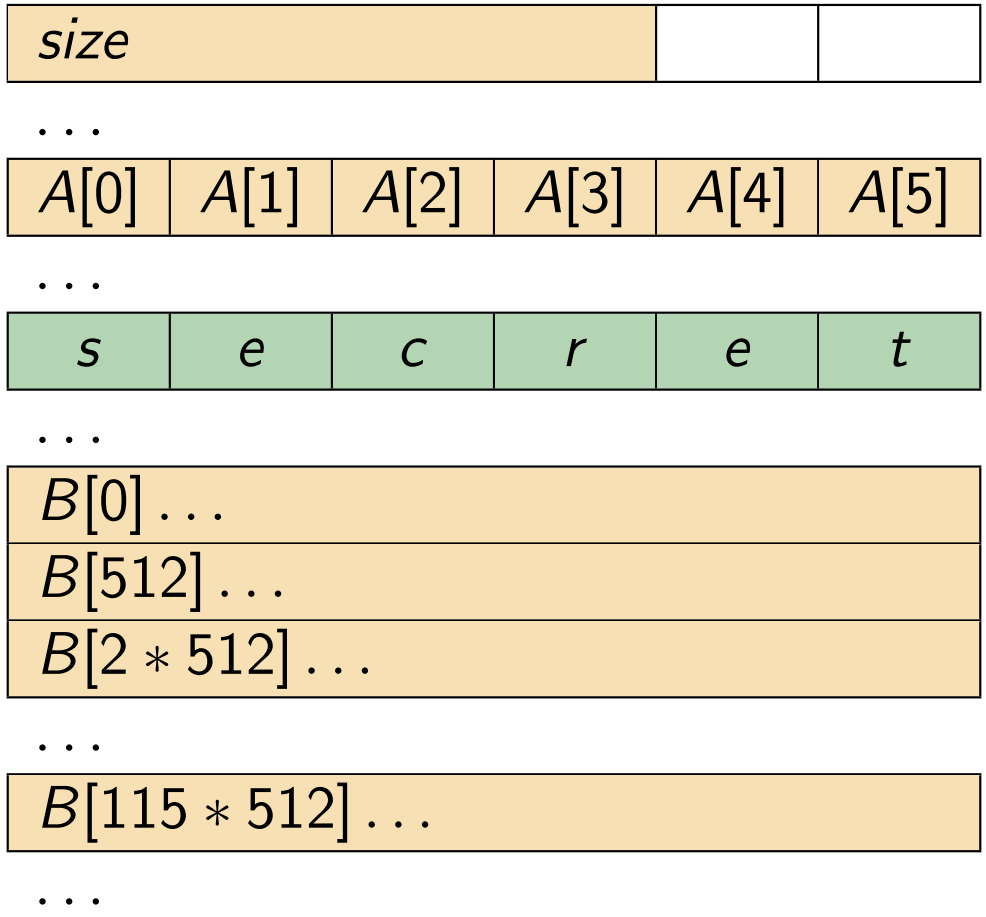
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Suppose

- `size` is not in the cache.
- `A` is not in the cache.
- `B` is not in the cache.
- *secret* is in the cache.
- Branch prediction expects `y<size` to be true.

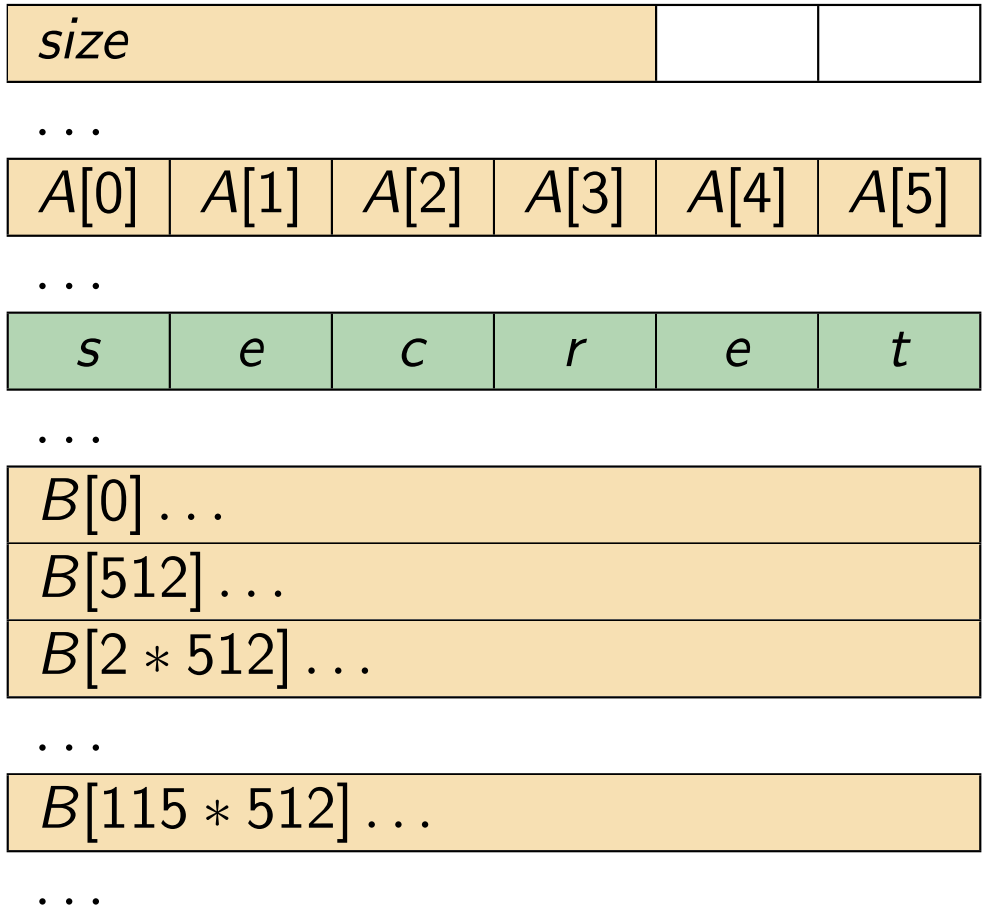


Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

- The intruder guesses y, so that $A + y$ is the address of secret.
  - ★ So: `y>=size`!

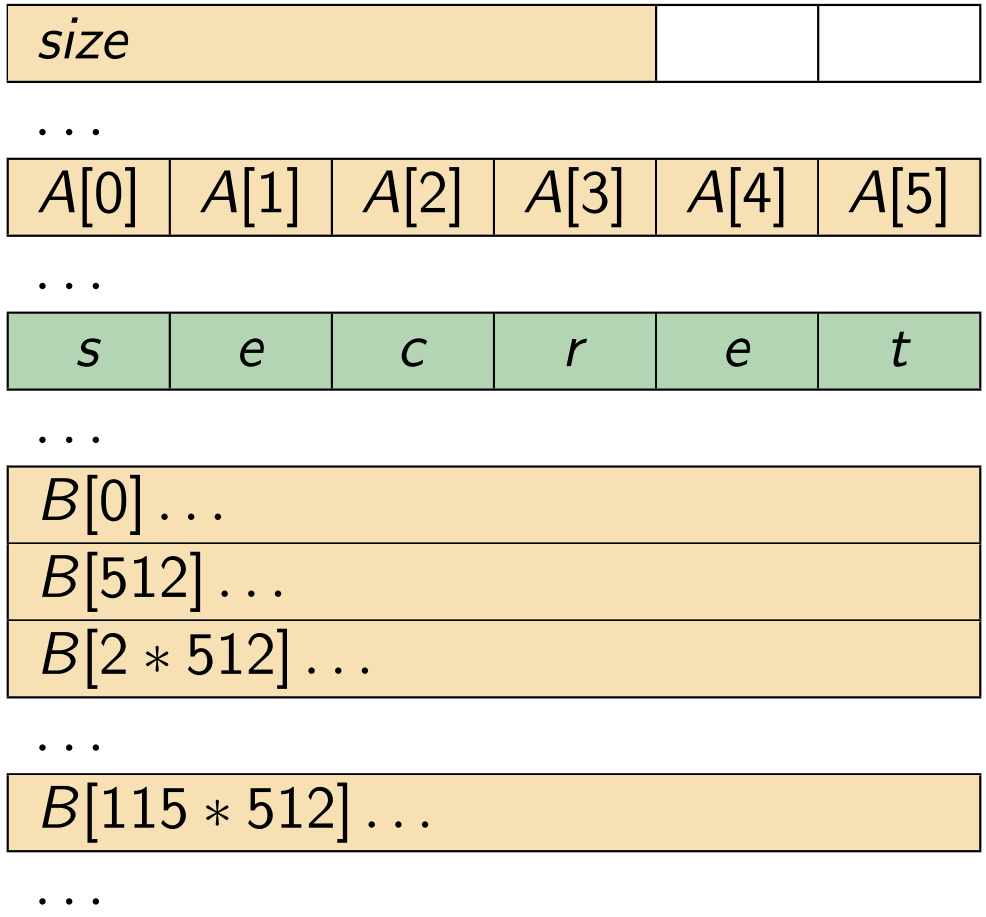


Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

- The intruder guesses y, so that $A + y$ is the address of secret.
  - ★ So: `y>=size`!
- Evaluate `y<size`:
  - ★ `size` not cached,
  - ★ branch prediction is positive

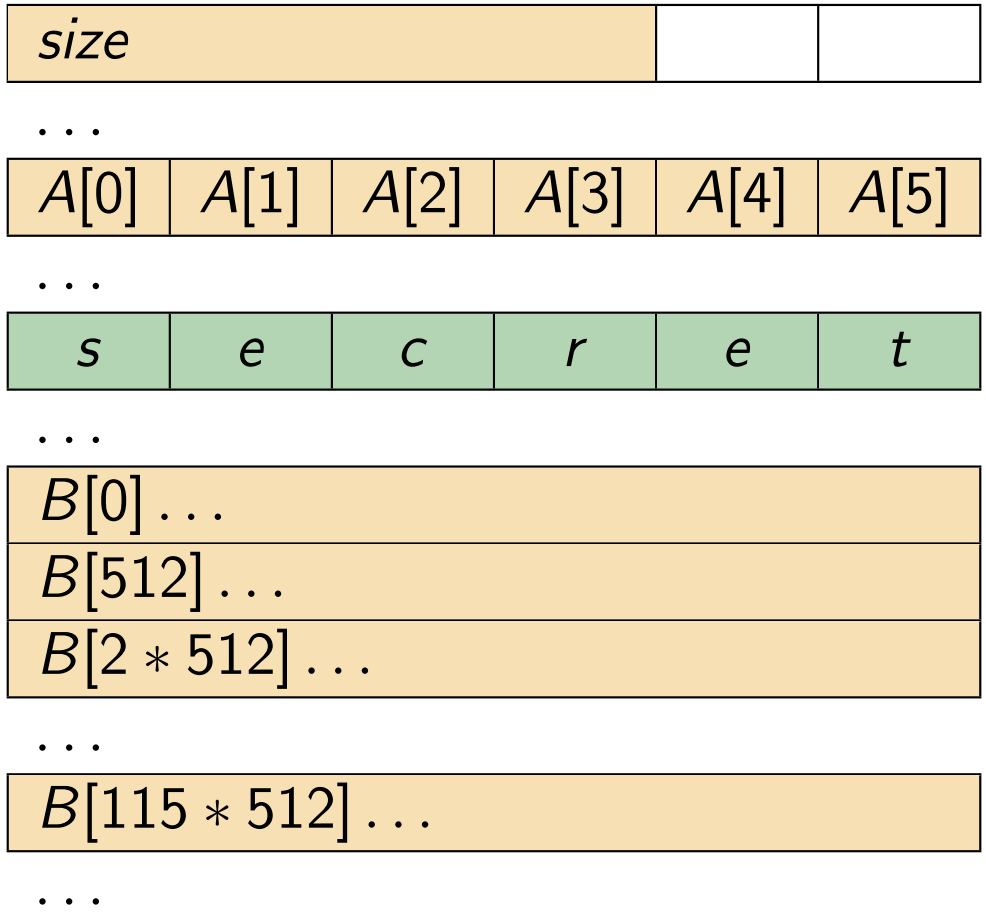


Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

- The intruder guesses y, so that $A + y$ is the address of secret.
  - ★ So: `y>=size`!
- Evaluate `y<size`:
  - ★ `size` not cached,
  - ★ branch prediction is positive



Thus start speculative execution of `B[A[y]*512]`.

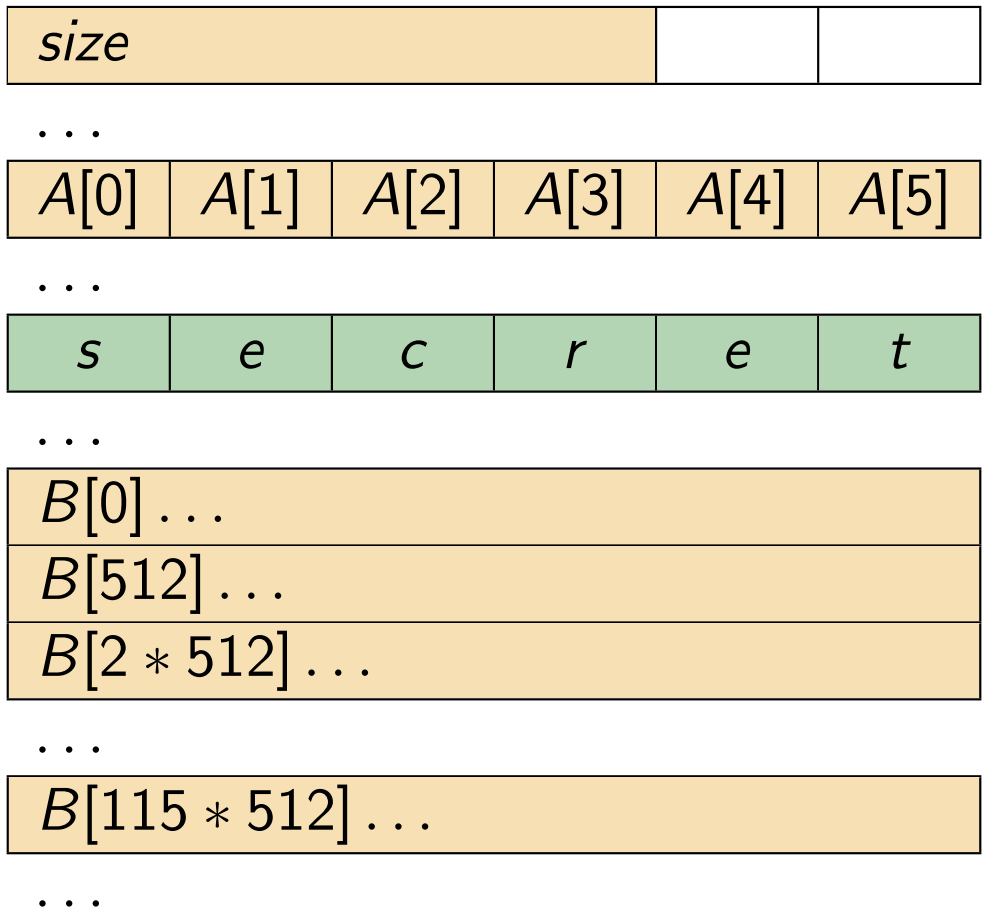
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Speculative execution of `B[A[y]*512]`

- $A + y$ is address of secret.



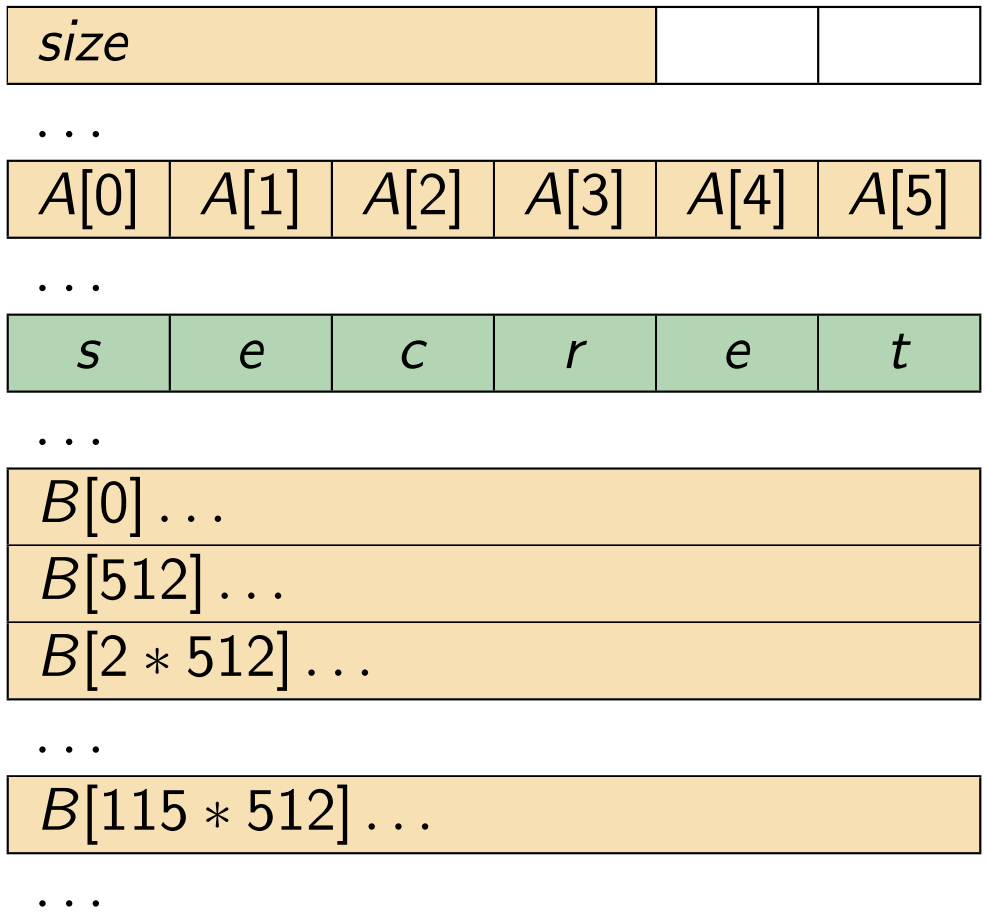
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Speculative execution of `B[A[y]*512]`

- $A + y$ is address of secret.
- So $A[y]$ is in the cache!



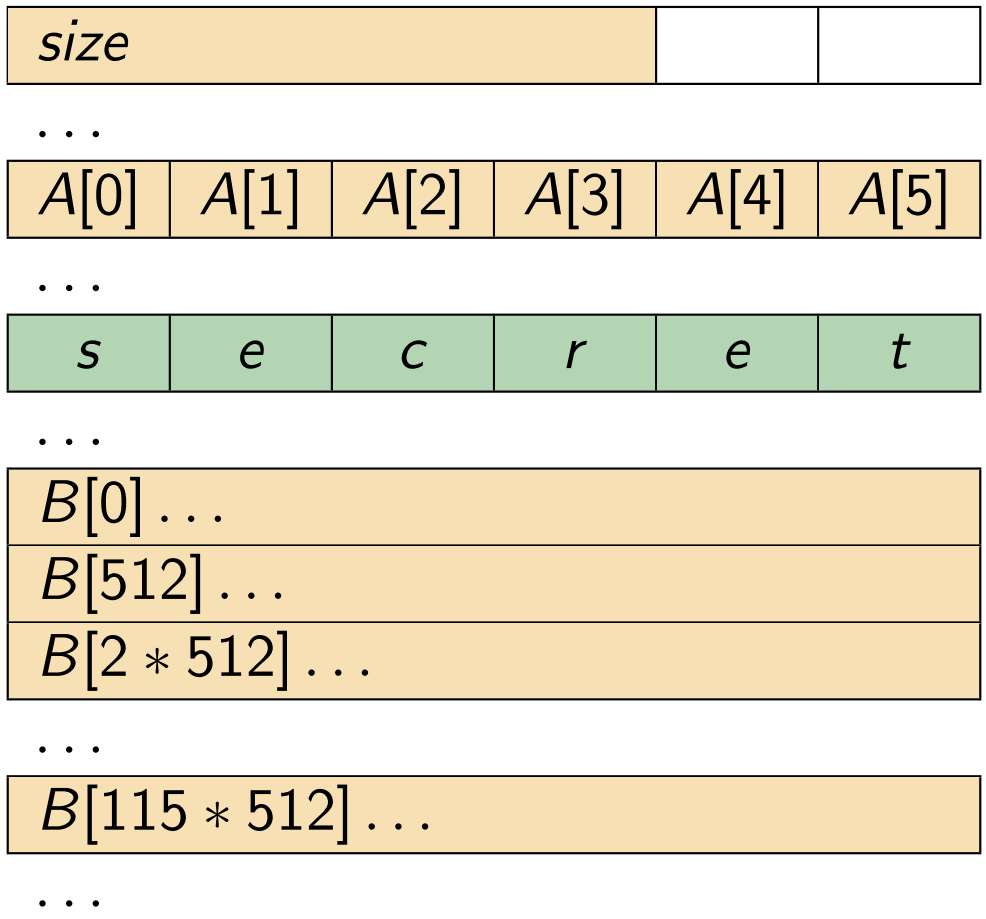
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Speculative execution of `B[A[y]*512]`

- $A + y$ is address of secret.
- So $A[y]$ is in the cache!
- So processor loads $B[A[y] * 512] = B[115 * 512]$



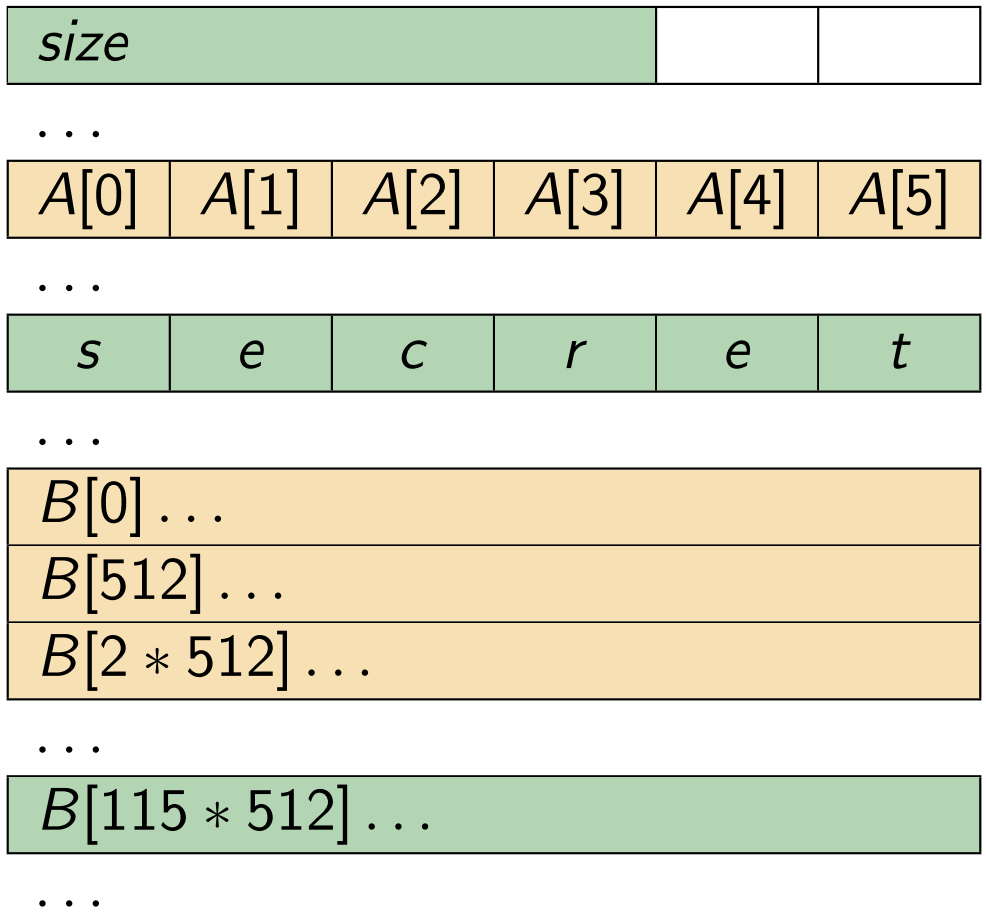
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Speculative execution of `B[A[y]*512]`

- $A + y$ is address of secret.
- So $A[y]$ is in the cache!
- So processor loads $B[A[y] * 512] = B[115 * 512]$
- ... it is now cached.



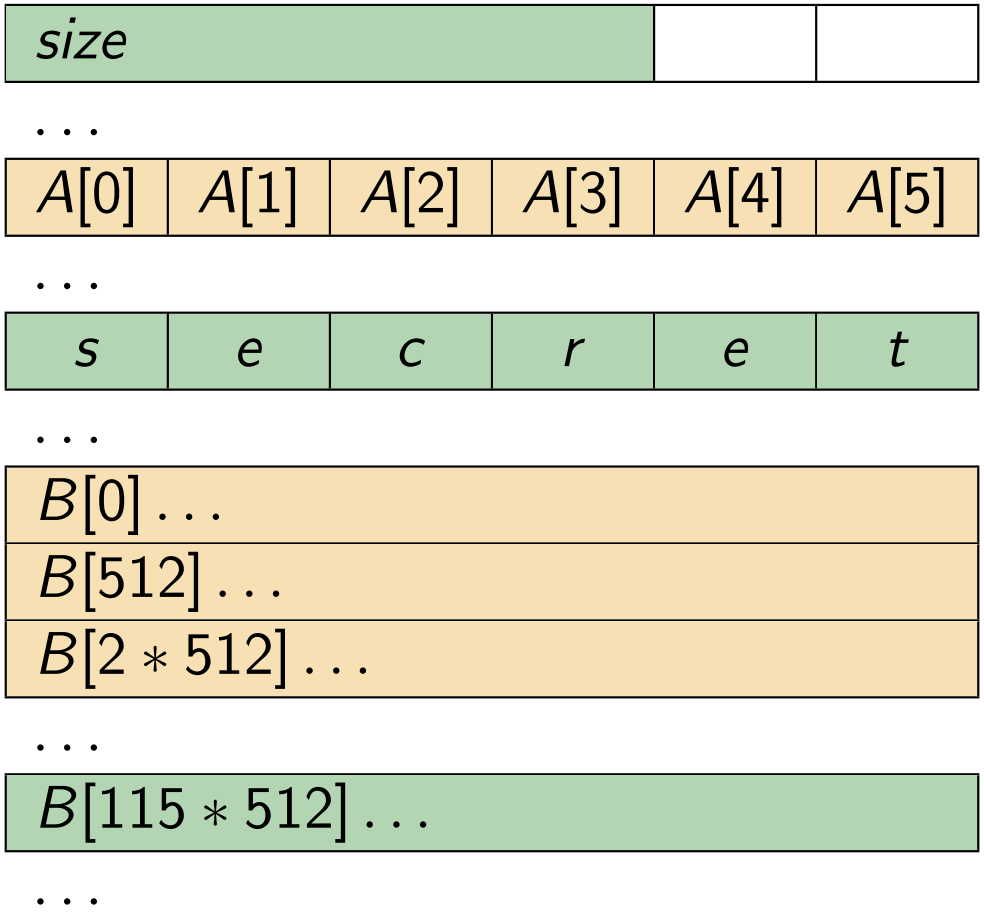
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Also `size` has arrived

- Thus, `y<size` can be computed to be false.
- Thus, the processor reverts to the state before.



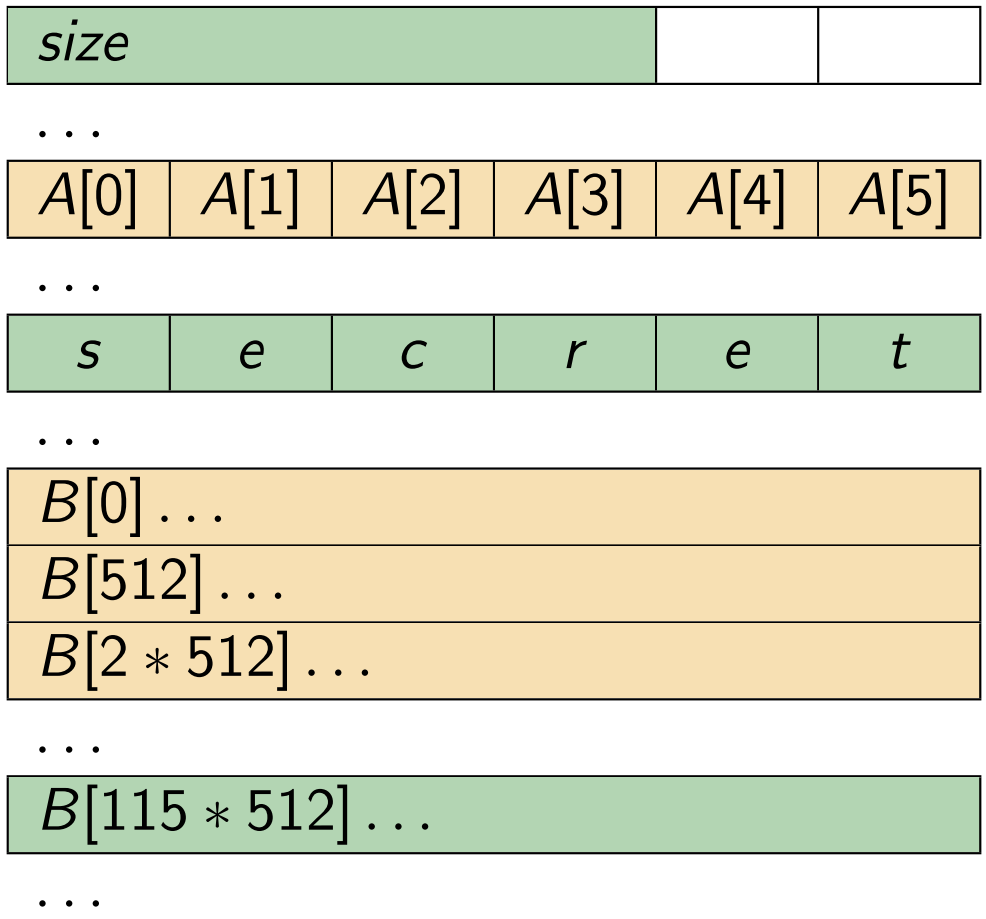
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Also `size` has arrived

- Thus, `y<size` can be computed to be false.
- Thus, the processor reverts to the state before.
- But the cache remains!



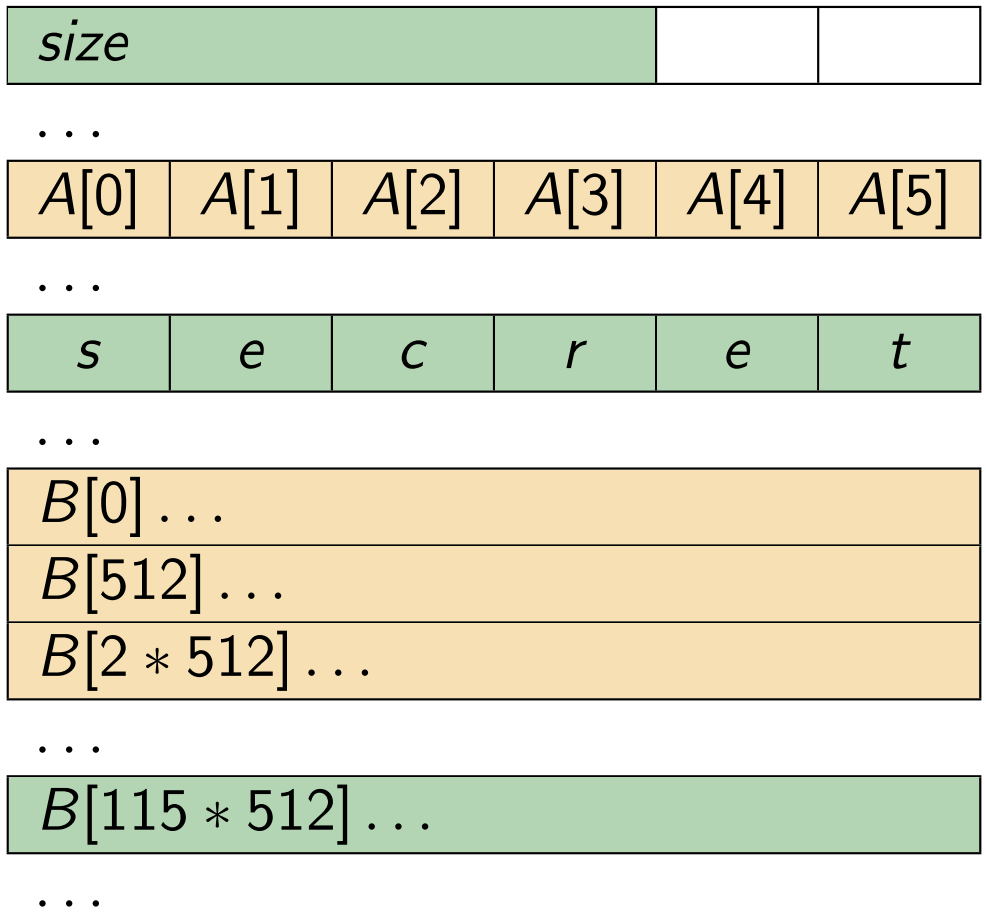
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Also `size` has arrived

- Thus, `y<size` can be computed to be false.
- Thus, the processor reverts to the state before.
- But the cache remains!



The intruder can now try to access `B[k*512]` for $k \in \{0..255\}$.

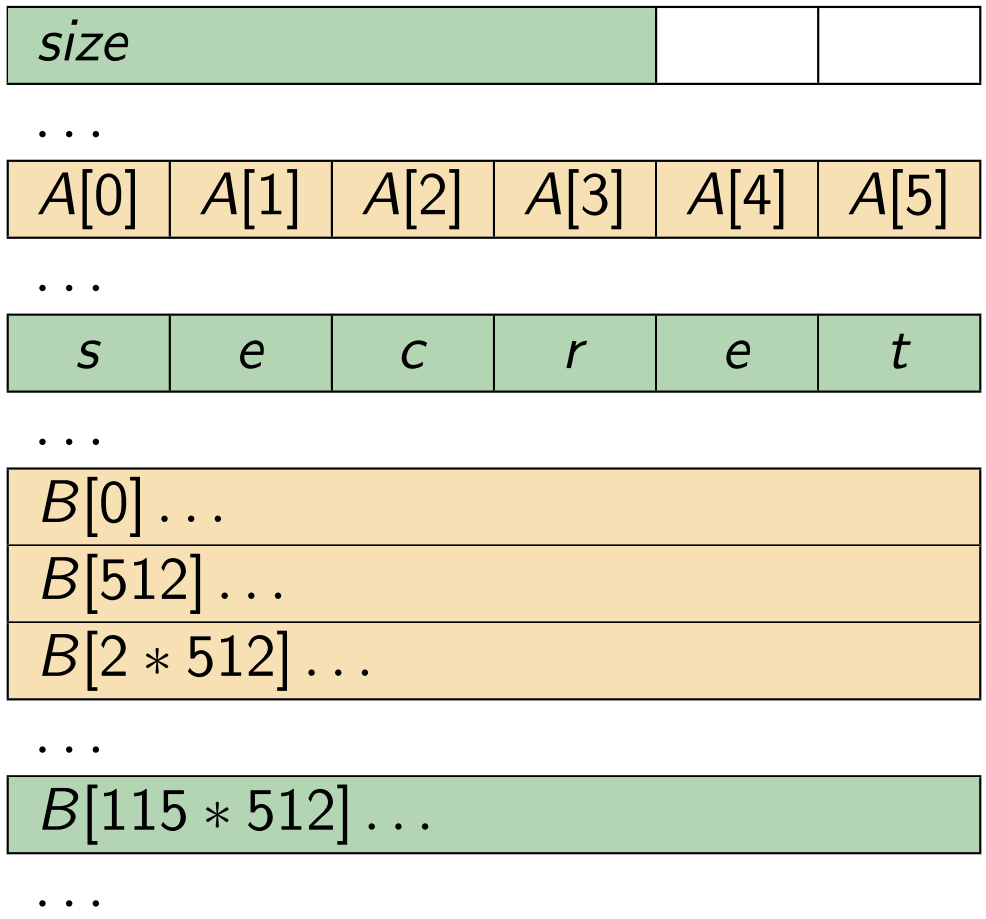
Spectre: Example

Victim code

```
if (y<size)
  temp &= B[A[y]*512]
```

Also `size` has arrived

- Thus, `y<size` can be computed to be false.
- Thus, the processor reverts to the state before.
- But the cache remains!



The intruder can now try to access `B[k*512]` for $k \in \{0..255\}$.
This is fast iff $k = 115$: he knows the first letter of the secret.

Speculative load hardening

The hardened code contains a few more operations:

```
        mov     size, %rax
        mov     y, %rbx
        mov     $0, $rdx
        cmp     %rbx, %rax
        jbe     END
        cmovbe  $-1, %rdx
        mov     A(%rbx), %rax
        shl     $9, %rax
        or      %rdx, %rax
        mov     B(%rax), %rax
        or      %rdx, %rax
        and     %rax, temp
END:    ...
```

Side Channel Analysis

- Side channels are usually leaks that the designers never have thought of.
- Many problems exist for years without getting noticed – how much else is out there?
- Very active research field:
  - ★ In order to prove a solution correct, one needs a precise model first.
  - ★ For instance Speculative non-interference and the Spectector tool. [2] Marco Guarnieri et al.: *Spectector: Principled Detection of Speculative Information Flows*, Security and Privacy, 2020.

References I


D. Brumley and D. Boneh.
Remote timing attacks are practical.
Comput. Networks, 48(5):701–716, 2005.


M. Guarnieri, B. Köpf, J. F. Morales, J. Reineke, and A. Sánchez.
Spectector: Principled detection of speculative information flows.
In *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*, pages 1–19. IEEE, 2020.


P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom.
Spectre attacks: exploiting speculative execution.
Commun. ACM, 63(7):93–101, 2020.